

2 – DATATYPES

When learning a new language, the first place to start is examining the primitive/native data-type supported by the language. A good understanding of datatypes is crucial in writing performant and robust code and it will allow you be more fluent in the language usage.

Unlike the other structured languages that you might have studied so far, Python is a dynamically-typed language (the other dynamically-typed language you are likely to encounter is Ruby or JavaScript™). This means that a variable may contain different data types in the life time of a program.

FYI

A statically typed language is a programming language where variable types are checked at compile-time (before the program runs), rather than at runtime.

In this chapter we will examine the primitive (basic data) types that are available in the normal python runtime such as **int**, **float**, **bool**, **str** and **none**. Non-primitive (composite data) types such as **set**, **list**, **tuple** and **dict** will be covered in more detail in subsequent chapters.

Python supports the following primitive data types:

2.a – None

None is a special constant, that is un-equal to everything except to another None. This is similar to the Null object in C#.

```
>>> None == 0
False
>>> None == []
False
>>> None == False
False
>>> x = None
>>> y = None
>>> x == y
True
```

It is an object that has its own datatype.

```
>>> type(None)
<class 'NoneType'>
```

2.b – Numerics

Numbers are immutable, this means that values in memory are not mutated, another spot is chosen with the new value.

There are three types of numbers available without any imports:

2.b.1 – Integers

Integers are whole numbers that have unlimited precision. You may specify an integer literal in base 10 (this is the default), base 2, base 8 or base 16 by add a prefix of `0b`, `0o` or `0x` respectively.

Fun fact

C# has 13 primitive numeric types

Some int literals

```
>>> 7                                #base 10 literal
7
>>> 0b100                            #base 2 literal (prefix 0b)
4
>>> 0o27                             #base 8 literal (prefix 0o)
23
>>> 0xf                             #base 16 literal (prefix 0x)
15
>>> 0b100_0000                       #binary literal with digit separator
64
>>> 100_000                          #decimal literal with digit separator
100000
>>> 12345678901234567890123456789012345678901234567890 + 1
12345678901234567890123456789012345678901234567891
```

The last but one int literal contains a **digit separator** (an underscore). Underscores are permitted anywhere in a **numeric** literal but not at the beginning or at the end. This is purely cosmetic, it does not affect the value of the number, it just makes it easier for the developer to read the value.

3.b.1.1 – bool

This is an int sub-type, having only two literal bool values: **True** or **False**

Fun fact

The objects **True**, **False** and **None** are considered immortal in python.

```
.  
>>> False + 0  
0  
>>> True + 0  
1  
>>> True + True  
2
```

Unlike C#, however any python object can be tested for truth value. The following values will be considered False:

- None,
- False
- Zero numeric type: 0, 01, 0.0, 0j
- Any empty sequence: '', {}, []

All other values are considered True

```
>>> bool([])           #empty list  
False  
>>> bool([0])  
True  
>>> bool('')           #empty string  
False  
>>> bool('0')  
True  
>>> bool(0)           #integer zero
```

```
False
>>> bool(3.1415)
True
```

2.b.2 – Floating Point Numbers

Floats are numbers with decimal places. This type is referred to floating point because it stored as a binary fractions and only fractions where the denominator is a power of 2 can be stored exactly.

When you do a calculation that results in a fraction e.g. $4 \div 3$ the result will always be a floating point number. Unlike C#, python only has a single float type.

You should avoid using float for high precision arithmetic as required for financial applications.

Fun fact

Most languages are not able to represent float value exactly. The Decimal type in the decimal module is able to represent a decimal number exactly.

```
10., .001, 1e100, 3.14e-10
3.141_592                #value of pi
5.972e24                 #mass of the earth
9.1093837e-31            #mass of an electron in kg
```

2.b.3 – Complex Numbers

This is very similar to the complex number in Pure Mathematics: there is a real and an imaginary part. In Python a literal of complex datatype must have at least a complex part. The following are some examples of complex literals:

Fun fact

The Italian mathematician **Gerolamo Cardano** first used complex number in the 16th century.

```
3.14j, 10.j, 10j, .001j, 1e100j, 3.14e-10j
1.414_213j                #root of -2
```

2.b.4 – Fraction

This section is for information only. The fraction datatype is available via an explicit import only. You are able to use numbers as ratios of integers.

2.b.5 – Decimal

This section is also for information only. The decimal datatype is available via an explicit import only.

2.c – Sequences:

The six sequences in python share some common operations such as iterable and indexable:

- Strings
- Tuples -> ()
- Lists -> []
- range -> range(5)
- stack, queue will not be covered in this course

~~Unlike most modern languages, python support negative indexing. If the index is positive, then it is interpreted (like other languages) position from the start of the sequence. If the index is negative, then position start from the end of the sequence. So, an index of -1 will reference the last item in the sequence.~~

2.c.1 – Strings

Strings are text delimited by a matching pair of single, double or triple quotes. In this course, we will use a single quote for strings with a few exceptions. Triple quotes (either single or double) are used for multi-line string such as docstring.

```
a = 'Hello '  
b = "world"  
c = '''This string  
can span multiple  
lines.'''
```

2.c.1.1 – F-Strings

You can insert a variable or an expression into a string using f-strings. The syntax is simple, *just insert the variable in-between a set of curly braces {}*.

```
days = 365
print(f'There are {days} in a year')
```

2.c.2 – Tuple

A sequence of immutable python objects. This means that it (any of the items) may not be changed after assignment. The items do not have to be the same type.

```
>> a = ()                                #verbatim definition
>> a
()

>> b = (1)                               #this will not do a single item tuple
>> b
1

>> b = (1, )                             #now you have a single item tuple
>> b
(1,)
```

2.c.3 – Range

A string is just a string of characters that must be delimited with either single, double or triple single or double quoted.

```
a = 'Hello '
```

2.c.4 – List

A sequence of mutable python objects. This is the work horse in python for collection. It serves the purpose of arrays in other languages and it may contain any type of members. The following statements will return lists:

```
>> a = []                                #verbatim definition
>> a
[]

>> b = 'Hello world'.split()            #list from calling split on a string
>> b
['Hello', 'world']
```

2.d – Mappings:

2.d.1 – Dictionary

A sequence of key-value pairs.

Each key must be unique and immutable such as strings, numbers, or tuples. A list may not be a key. There is no restriction on the value, it may be any python object. Dictionary does not support positional indexing, only key-indexing.

This data type is flexible and powerful and is extensively used in the language and will be heavily used in this course.

```
#there are lots of ways of creating a dictionary
>>> a = {'red': 'Danger', 1: 'first', 'second': 2, ('a', 'set'): 'a tuple
is my key'}

>>> b = dict(zip(['ilia', 'hao', 'yin'], ['Ilia Nika', 'Hao Lac', 'Yin
Li']))

>>> c = dict([('arben', 'Arben Tapia'),('patrick','Patrick Gignac')])

>>> d = dict(nar='Narendra Pershad', joanne='Joanne Filotti')
```

2.e – Set types

Finite un-ordered collection of unique items. They can not be indexed by a subscript i.e., they are not indexable. However, they can be iterated over i.e., they are iterable. Common uses for sets are fast membership testing, removing duplicates from a sequence, and computing mathematical operations such as intersection, union, difference and symmetric difference.

Set (mutable) and Frozen sets (immutable).

2.e.1 – Set

A sequence of unique immutable python objects. Sets, list or dict cannot be in a set. Set does not support indexing.

Mimics a Mathematical set such that no duplicates are allowed.

2.f – Binary types

These types are used to handle binary data. They include the following:

```
>>> a = b'ABC'                                #immutable sequence of bytes
>>> print(a)
b'ABC'
>>> b = bytearray([65, 66, 67]) #mutable sequence of bytes
>>> print(b)
bytearray(b'ABC')
```

2.g – Enum

This datatype is available via an explicit import. We will not be using this type in Programming 1.

2.h – Checking Data Types

You can use the `type()` function to check what is the data type of a particular variable.

```
>>> type('Hello')
<class 'str'>
>>> type(3)
<class 'int'>
>>> type(True)
<class 'bool'>
>>> type(3.141592)
<class 'float'>
>>> type([])
<class 'list'>
>>> type(())
```



```
<class 'tuple'>
>>> type({1,})
<class 'set'>
>>> type({})
<class 'dict'>
```

2.i – Converting Data Types

Most languages provide a mechanism to convert from one data type to another type either through implicit or explicit casting. You can convert a variable from one data type to another.

2.i.1 – Implicit

This kind of conversion is automatic e.g.:

```
a = 3.15                #this is a float
b = 5                   #this is an int
c = a + b               #b is up cast to a float before addition

#the following does not work (it does work in C#)
a = 'Hello'             #this is a str
b = 5                   #this is an int
c = a + b               #this does not work
```

2.i.2 – Explicit

Explicit casting is done by constructor methods such as `int()`, `float()` and `str()`:

2.i.2.1 – Converting to string:

```
str()
n = 354
print(str(new_n))      #result 354
```

2.i.2.2 – Converting to int:

```
>>> int(3.14)           #truncates down the float
3
>>> int(-1)
-1
>>> int(0b110)          #binary literal
6
>>> int('110', base=2)
6
>>> int('fe', base=16)
254
>>> int('11', base=12)
13
```

This converts a number or string to an integer assuming base 10. If you would like to specify an alternate base, then the argument must be a string. The string literal must be a legal string: 'one' is not a legal number and will cause a `ValueError` to be raised.

2.i.2.3 – Converting to float:

```
>>> float(-1)
-1.0
>>> float('3.1415')
3.1415
>>> float(1e8)
100000000.0
>>> float(1.2e-3)
0.0012
```

Other conversions can be done with: `tuple()`, `list()`, `set()`, `dict()`, `ord()`, `chr()`, `hex()`, `oct()`, `bin()`

2.j – Operators

Operators usually takes two operands to create expressions. This expression is evaluated to gives a result. The operator determines the type of operands as well as the resulting value. Unary operators only require a single operand whilst ternary operators require three operands.

2.j.1 – Assignment Operators

This operator binds a value to a memory location (name). i.e it sets the value of a variable. The left-hand side value of the operator is bound to the variable on the right-hand side. If the left side is an expression, then it is evaluated and the resulting value is set to the variable on the right-hand side.

```
>>> a = 5          #a is 5

>>> a = 4 * 3      #a is 12

>>> a *= 3         #a is 3 times a or 36

>>> a = b = c = 0  #a, b, and c is set to 0

>>> a, b, c = 0, 1, 2    #a is 0, b is 1 and c is 2

>>> a, b = b, a      #values of a and b are swapped

>>> a = [0, 1, 2, 3, 4]
>>> b, *c = a
>>> b
0
>>> c
[1, 2, 3, 4]

>>> *b, c = a
>>> b
[0, 1, 2, 3]
>>> c
4
```

2.j.1 – Arithmetic Operators

You can do mathematical calculations with Python as long as you know the right operators. The order of precedence of the operator is listed in order.

```
>>> 5 ** 2          #exponent
25
>>> 4 * 3           #multiply
12
>>> 18 / 7           #divide
2.5714285714285716
>>> 18 // 7          #integer divide
2
>>> 18 % 7           #modulus
4
>>> 2 + 7            #add
9
>>> 9 - 4            #subtract
5
```

2.j.1.1 – The Modulo Operator

Often, you'll want to know what is the remainder after a division. e.g. $4 \div 2 = 2$ with no remainder but $5 \div 2 = 2$ with 1 remainder. The modulo does not give you the result of the division, just the remainder. It can be really helpful in certain situations, e.g. figuring out if a number is odd or even.

```
5 % 2                #result is 1
```

2.j.2 – Relational (Comparison) Operators

These operators are normally used to compare two values of the same type. The result is a bool value. You may also use these operators with string type values.

```
>>> 3 == 4           #equal
False
>>> 3 != 4           #not equal
```

```
True
>>> 3 > 4          #greater than
False
>>> 3 < 4          #less than
True
>>> 5 >= 3         #greater than or equal to
True
>>> 5 <= 3         #less than or equal to
False
```

2.j.3 – Logical Operators

Use to compare two values normally of the same type. The result is a bool. You may also use these operators with string type values.

```
>>> True and True  #True only if both operands are True
True
>>> True and False
False
>>> True or True
True
>>> True or False
True
>>> False or False #False only if both operands are False
False
>>> not True        #inverts the value of the single operand
False
>>> not False
True
```

2.j.4 – Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location.

```
>>> a = [3]
>>> b = [3]
```

```
>>> a == b          #compares the values
True
>>> a is b          #compares the memory addresses
False
>>> id(a)
2354776320832
>>> id(b)
2354776273920
```

2.j.5 – Membership Operators

This can be used to test if a value is present in a sequence (string, list, tuple or dict).

```
>>> 'a' in 'Narendra Pershad'
True
>>> 'q' in 'Narendra Pershad'
False
>>> 'hello' in ['hello', 'world']
True
>>> 1 in {'one': 1, 'two': 2, 'three': 3}
False
>>> 1 not in {'one': 1, 'two': 2, 'three': 3}
True
>>> 'one' in {'one': 1, 'two': 2, 'three': 3} #for dict, the in operator
only check the keys
True
```

2.j.6 – Bitwise Operators

This category of operators requires two integer operands (except ~ not) and it returns an integer. It operates at the bit-level, so the results are not immediately understandable. However, examining the results (and operands) as binary value will make the operators more palatable.

```
>>> 9 << 2
36
>>> print(f'{bin(0b01001 << 2)}')
0b100100
```

```
>>> print(f'{bin(0b01001 << 3)}')
0b1001000
>>> print(f'{bin(0b01001 >> 2)}')
0b10
>>> print(f'{bin(0b01001 & 0b00101)}')
0b1
>>> print(f'{bin(0b01001 | 0b00101)}')
0b1101
>>> print(f'{bin(0b01001 ^ 0b00101)}')
0b1100
>>> print(f'{bin(~0b01001)}')
-0b1010
```

2.j.6.1 – The augmented Operator

Augmented assignment provides a short way to perform a binary operation (arithmetic, boolean or bitwise) and assigning the results back to one of the operands. You can also use any of the other mathematical operators e.g. -=, +=, *=, /=, //=, **=, |=, >>=, <<=, %=, or ^=

```
my_number = 4
my_number += 2      #my_number is now is 6
```

2.j.7 – Operator precedence

If you have an expression with more than one operator, then python uses a set of internal rules to decide the order that the computation must follow. The table below list the various operators in descending order of precedence.

Operator	Description
()	Parentheses
**	Exponentiation
+x -x ~x	Unary plus, unary minus, and bitwise NOT
* / // %	Multiplication, division, floor division, and modulus

+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
 	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators
not	Logical NOT
and	AND
or	OR